ASSIGNMENT 1

TOPIC: PROJECT OVERVIEW, REQUIREMENTS & IMPLEMENTATION ARCHITECTURE

1. Describe your project.

Project Title: *College AI Chatbot* is an automated, intelligent conversational assistant and campus self-service knowledge portal designed and developed specifically for **G H Raisoni International Skill Tech University, Pune**. The primary objective is to provide students, parents, and administrative staff with instant, round-the-clock (24/7) automated responses to frequently asked institutional queries spanning admissions, fee structure, semester examination timetables, 4-tier attendance detention rules, syllabus/course details, library timings, faculty contact directories, hostel/mess regulations, campus placement track records, and location transit directions for both Wagholi and Yerawada Pune campuses.

The system employs a hybrid natural language processing engine combining exact pattern matching with typo-tolerant fuzzy matching (via Python's difflib.SequenceMatcher) over an indexed relational database. It incorporates authenticated student access where enrolled students log in via their institutional registration numbers (e.g., ISTU25010120001) to fetch real-time personalized records including individual attendance percentage, 4-tier detention risk tier, pending fee balance, cumulative CGPA, assigned faculty mentor, and hall ticket eligibility. Furthermore, the application features bi-directional voice accessibility via Web Speech API (Speech-to-Text and Text-to-Speech audio readout), Progressive Web App (PWA) offline-ready installability, and an administrative analytics command center with live KPI metric cards and CSV audit logging.

2. Advantages of your project.

- **24/7 Instant Self-Service:** Eliminates long student queues at administrative and departmental enquiry counters by offering sub-second automated answers outside office hours and during exam seasons.
- **Typo-Tolerant Conversational Robustness:** Accommodates student spelling errors, typos, and phonetic variants (e.g., “hostal fee”, “atendance rule”) using dual-tier fuzzy matching without throwing jarring syntax errors.
- **Personalized Academic & Detention Insights:** Automatically computes student attendance ratios and classifies them into institutional standing tiers (Good Standing ≥ 75%, Warning 60–74.9%, Critical 50–59.9%, Course Detention < 50%), providing actionable remediation guidance.
- **Multi-Modal Voice Accessibility:** Allows hands-free operation through microphone Speech-to-Text (STT) and synthesized spoken voice output (TTS) powered by browser Web Speech API and Google Text-to-Speech (gTTS).
- **Administrative Control & CSV Audit Exports:** Enables non-technical staff to update FAQs dynamically, review satisfaction feedback ratings (👍/👎), and export query logs to CSV for compliance audits.
- **PWA Native Experience with Zero Licensing Overhead:** Operates as an installable app across smartphones, tablets, and desktops without third-party app store commissions, built entirely on an open-source technology stack.

3. How do you gather the requirements of your project?

Requirements engineering followed an empirical, multi-stage stakeholder elicitation process:

a) Enquiry Counter Observation & Student Surveys: Documented repetitive inquiries asked by students and visitors at GHRISTU administrative desks over past semesters. These high-frequency inquiries were organized into 14 distinct categories (Admissions, Fees, Timetable, Attendance, Library, Courses, Faculty, Branch Change, Placements, Hostel, Scholarships, Clubs, Helplines, Campus Transit).

b) Institutional Policy Review: Examined official university guidelines regarding course registration, 75% mandatory attendance policies, examination hall ticket eligibility, and administrative escalation workflows.

c) Mentor Guidance & Prototype Reviews: Consulted project guide **Asst. Prof. Sheela Satav** to identify non-functional requirements including responsive theme switching (Light/Dark mode), cryptographic password hashing (PBKDF2/scrypt), database normalization (3NF), and automated unit test verification.

4. How and where do you implement your project?

Implementation Stack: Developed using Python 3 (Flask RESTful framework) for backend API routing, SQLite3 for relational persistence, and HTML5, modern CSS3 (Flexbox/CSS Grid), and asynchronous vanilla JavaScript (ES6+ fetch() API) for frontend user interface rendering.

Deployment Environment: Source code is version-controlled via Git and hosted in a public GitHub repository (kanishksingh-01/college-ai-chatbot). It is deployed live on **Render Cloud Platform** using continuous deployment (CI/CD) pipelines, accessible globally at: https://college-ai-chatbot-c94w.onrender.com. Every code push to the main branch automatically triggers automated builds, dependency resolution via requirements.txt, and zero-downtime service redeployment.

5. Which software you used in your project?

- **Visual Studio Code (VS Code):** Primary Integrated Development Environment (IDE) used for code authoring, linting, debugging, and terminal execution.
- **Python 3.10+:** High-level programming language executing the server runtime, natural language processing, and database transaction logic.
- **Flask (v3.0.3):** Lightweight, WSGI web application framework used to construct RESTful endpoints (/chat, /login, /admin, /export).
- **SQLite 3:** Serverless, zero-configuration relational database engine embedded directly into the application for storing institutional FAQs, student profiles, and chat logs.
- **Werkzeug (v3.0.3):** Cryptographic security module handling secure password hashing using salt-extended PBKDF2 with SHA-256 and scrypt algorithms.
- **Web Speech API:** Native browser APIs (webkitSpeechRecognition and SpeechSynthesis) for real-time speech-to-text and text-to-speech audio rendering.
- **Git & GitHub:** Distributed version control system and cloud repository hosting for collaborative commit tracking and continuous delivery.
- **Render Cloud Hosting:** Fully managed cloud application hosting platform running Linux container environments for public internet availability.
- **Google Chrome & Safari:** Web browsers utilized for cross-platform debugging, DevTools performance auditing, responsive testing, and PWA manifest validation.

6. What methodology is used for the project?

The project was executed using the **Iterative and Incremental Software Development Life Cycle (SDLC) Model**. Rather than attempting a rigid one-pass deployment, the chatbot was architected across 6 progressive iteration cycles:

• *Iteration 1 (Core Prototype):* Console-based keyword matching algorithm and baseline response dictionary.
• *Iteration 2 (Persistence & Web Interface):* SQLite database schema creation (3NF) and responsive HTML/CSS/JS chat screen.
• *Iteration 3 (Student Authentication):* Student portal login enabling personalized queries for attendance percentage, fees, CGPA, and mentor.
• *Iteration 4 (Administrative Governance):* Password-protected admin command center with live KPI metric cards and CSV audit logging.
• *Iteration 5 (Accessibility & PWA):* Web Speech API voice interaction (STT/TTS), Dark/Light mode theme toggle, and PWA Service Worker caching.
• *Iteration 6 (Testing & Cloud Deployment):* Automated unit testing suite (test_suite.py), Render CI/CD cloud deployment, and system verification.

Each iteration underwent thorough functional testing and verification before incorporating subsequent modules, ensuring high system stability and zero regression flaws.

7. What is the hardware or software of the project?

Hardware Requirements:

• *Client Device:* Any standard PC, laptop, tablet, or smartphone equipped with at least 1 GB RAM, internet connectivity, microphone (for voice query input), and audio speakers/headphones (for spoken feedback).
• *Cloud Server:* Render Linux Cloud Container provisioned with 512 MB RAM, 0.1 vCPU (shared), and persistent SSD block storage.

Software Requirements:

• *Client-Side:* Modern web browser supporting HTML5, CSS3, ES6 JavaScript, and Web Speech API (Google Chrome 90+, Safari 14.1+, Microsoft Edge 90+, Mozilla Firefox 88+).
• *Server-Side:* Linux OS, Python 3.10 runtime environment, Flask WSGI framework, SQLite3 database, and Gunicorn web server.

8. What is your role in the project?

The project was developed collaboratively by a dedicated two-member academic team:

• **Kanishk Singh (Roll No: ISTU25010120001):** Lead Backend Developer & System Architect — Responsible for designing the Flask application structure, NLP query matching algorithms, fuzzy similarity logic (difflib), normalized 3NF database schema, student personalized data retrieval, automated test suite engineering (test_suite.py), Git version control workflow, and Render cloud deployment pipeline.
• **Vibhor Saini (Roll No: ISTU25010120002):** Frontend & UI/UX Specialist — Responsible for responsive client-side interface development, Light/Dark theme CSS engineering, Web Speech API integration (voice recognition and text-to-speech audio chimes), admin dashboard UI design, and manual cross-device testing.

9. Explain the data flow of your application.

The data flow follows a synchronous client-server request-response pattern:

1. **Input Capture:** The user enters a question via keyboard typing, selects a quick-reply category button, or speaks through the microphone (which converts speech to text via webkitSpeechRecognition).
2. **HTTP Request:** JavaScript packages the input into a JSON payload { message: "...", user_id: "..." } and sends an asynchronous HTTP POST request to the Flask /chat endpoint.
3. **Normalization & Intent Analysis:** The backend strips special characters, normalizes tokens to lowercase, and checks if the student is authenticated and requesting personal data (e.g., “my attendance”). If so, it queries the students table directly.
4. **Fuzzy Keyword Matching:** For general institutional FAQs, match_query() computes token overlap scores and similarity ratios against the college_info table to determine the optimal response.
5. **History Logging:** The user query, matched response, timestamp, and active session identifiers are committed to the chat_history table.
6. **Client Rendering:** The JSON response returns to the browser. JavaScript dynamically updates the DOM, renders tabular responses as structured HTML tables, and triggers the Web Speech synthesis engine to speak the answer aloud.

10. What are the entities in your project?

The system models four core database entities normalized in Third Normal Form (3NF):

- **Student Entity (students table):** Models enrolled student records. Attributes: roll_no (Primary Key, e.g., 'ISTU25010120001'), name, attendance (percentage real), fees_due (numeric balance), exam_date, course (e.g., 'BBA CA'), semester ('Semester III'), cgpa, and mentor (assigned faculty mentor).
- **CollegeInfo Entity (college_info table):** Represents the centralized institutional knowledge base. Attributes: id (Primary Key, autoincrement), category (one of 14 domains, e.g., 'Admissions', 'Fees'), keywords (comma-separated indexing tokens), and answer (formatted response text).
- **ChatHistory Entity (chat_history table):** Tracks conversational audit trails. Attributes: id (Primary Key), roll_no (Foreign Key referencing students), user_query, bot_response, timestamp, and feedback (satisfaction rating: thumbs up / thumbs down).
- **Admin Entity (admin table):** Manages administrative access control. Attributes: username (Primary Key), password_hash (salted PBKDF2/scrypt cryptographic hash), role, and session_state.

11. What is the drawback of your system?

- **Closed-Domain Rule Knowledge:** Because it operates on deterministic keyword and fuzzy matching rather than unconstrained large language models, the chatbot cannot synthesize novel answers for topics not pre-configured in the college_info database.
- **Administrative Maintenance Dependency:** Institutional accuracy relies upon administrative staff keeping semester dates, fee circulars, and faculty contact lists updated inside the admin portal.
- **Free Cloud Tier Cold-Start Latency:** When hosted on Render's free tier, server instances spin down during periods of 15-minute inactivity, requiring approximately 30 to 40 seconds to “wake up” on the initial cold request.
- **SQLite Concurrency Limitations:** SQLite employs file-level database locking during write transactions, which is sufficient for college-scale traffic but would require migration to PostgreSQL for massive enterprise loads.

12. How do you test your application?

Testing was conducted through a combination of automated and manual verification strategies:

• **Automated Unit Testing:** Implemented a test suite using Python's standard unittest framework (test_suite.py). Automated tests verify keyword scoring heuristics, fuzzy matching thresholds (confirming “hostal fee” resolves accurately to the Hostel category), personalized student queries (attendance, CGPA, mentor lookup), and HTTP response status codes with a 100% pass rate.

• **Manual Functional & UI Testing:** Conducted cross-device manual testing on desktop and mobile browsers (Chrome, Safari, Edge). Verified voice input transcription accuracy in noisy ambient conditions, dark mode contrast compliance, student authentication failure handling, administrative FAQ CRUD operations, and CSV export integrity.

• **Regression Testing:** Executed end-to-end user journeys on the live Render cloud deployment after every Git commit to confirm zero broken features.

13. From where do you get reference, guide for your project work?

Academic Guidance: Conceptual architecture, software engineering methodologies, UML modeling rules, and syllabus compliance were guided under the direct supervision of project guide **Asst. Prof. Sheela Satav**, Department of Computer Application, G H Raisoni International Skill Tech University, Pune.

Technical Documentation & Reference Sources:

• Official Flask Documentation (Pallets Projects) for WSGI routing, application context, and session management.

• Python Standard Library Documentation for difflib.SequenceMatcher, sqlite3, and unittest modules.

• MDN Web Docs (Mozilla Developer Network) for Web Speech API specifications (SpeechRecognition, SpeechSynthesis) and PWA Service Worker caching strategies.

• Software Engineering Literature: *Software Engineering: A Practitioner's Approach* by Roger S. Pressman; *UML Distilled* by Martin Fowler.

14. In the future, what changes will you make in your application?

- **Generative AI & RAG Architecture:** Integrate Google Gemini API / OpenAI API with Retrieval-Augmented Generation (RAG) over official college PDF syllabi and prospectus documents for contextual reasoning.
- **Multi-Channel WhatsApp & Telegram Bot:** Integrate official WhatsApp Business API to allow students to query the chatbot directly from their messaging apps without opening a browser.
- **Multi-Lingual Regional Support:** Add native voice and conversational support for Marathi and Hindi alongside English to support regional diversity.
- **ERP & RFID Live Synchronization:** Establish live webhook integration with the university's central ERP database for real-time lecture attendance biometric sync and automated online fee payment receipt generation.